

Qualité de code en C

Bonnes pratiques, robustesse et compilation stricte

Document pédagogique complet

7 avril 2026

Table des matières

1	Introduction	4
1.1	Objectif du document	4
1.2	Criticité	4
1.3	Contenu du document	4
1.4	Approche pédagogique	4
2	Compilation stricte (GCC)	5
2.1	Mauvais code	5
2.2	Bon code	5
2.3	Explication	5
3	Programme principal correct	6
3.1	Mauvais code	6
3.2	Bon code	6
3.3	Explication	6
4	Variables globales inutiles	7
4.1	Mauvais code	7
4.2	Bon code	7
4.3	Explication	7
5	Variables non initialisées	8
5.1	Mauvais code	8
5.2	Bon code	8
5.3	Explication	8
6	Valeurs de retour ignorées	9
6.1	Mauvais code	9
6.2	Bon code	9
6.3	Explication	9
7	Entrées utilisateur avec scanf	10
7.1	Mauvais code	10
7.2	Bon code	10
7.3	Explication	10
8	Conversion robuste avec strtoul	11
8.1	Mauvais code	11
8.2	Bon code	11
8.3	Explication	11

9	Overflow signé	12
9.1	Mauvais code	12
9.2	Bon code	12
9.3	Explication	12
10	Conversions signé / non signé	13
10.1	Mauvais code	13
10.2	Bon code	13
10.3	Explication	13
11	Masquage de variable	14
11.1	Mauvais code	14
11.2	Bon code	14
11.3	Explication	14
12	Prototypes de fonctions	15
12.1	Mauvais code	15
12.2	Bon code	15
12.3	Explication	15
13	Strict prototypes	16
13.1	Mauvais code	16
13.2	Bon code	16
13.3	Explication	16
14	Affectation au lieu de comparaison	17
14.1	Mauvais code	17
14.2	Bon code	17
14.3	Explication	17
15	Switch incomplet	18
15.1	Mauvais code	18
15.2	Bon code	18
15.3	Explication	18
16	Accès hors bornes dans les tableaux	19
16.1	Mauvais code	19
16.2	Bon code	19
16.3	Explication	19
17	Pointeurs nuls	20
17.1	Mauvais code	20
17.2	Bon code	20
17.3	Explication	20
18	Allocation dynamique	21
18.1	Mauvais code	21
18.2	Bon code	21
18.3	Explication	21

19	Erreur classique avec sizeof(pointer)	22
19.1	Mauvais code	22
19.2	Bon code	22
19.3	Explication	22
20	Macros non sûres	23
20.1	Mauvais code	23
20.2	Bon code	23
20.3	Explication	23
21	Usage inutile de goto	24
21.1	Mauvais code	24
21.2	Bon code	24
21.3	Explication	24
22	Commentaires utiles	25
22.1	Mauvais code	25
22.2	Bon code	25
22.3	Explication	25
23	Paramètre inutilisé	26
23.1	Mauvais code	26
23.2	Bon code	26
23.3	Explication	26
24	Nommage cohérent	27
24.1	Mauvais code	27
24.2	Bon code	27
24.3	Explication	27
25	Style de code cohérent	28
25.1	Mauvais code	28
25.2	Bon code	28
25.3	Explication	28
26	Conclusion	29

Chapitre 1

Introduction

1.1 Objectif du document

Ce document a pour objectif de présenter les bonnes pratiques essentielles en langage C à travers une approche comparative entre **mauvais** et **bon code**. Il vise à améliorer la qualité, la robustesse et la maintenabilité des programmes en mettant en évidence les erreurs courantes et leurs corrections.

1.2 Criticité

Le langage C est puissant mais peu permissif : il laisse au développeur la responsabilité de la gestion mémoire, des types et du contrôle des erreurs. Une mauvaise pratique peut rapidement conduire à :

- des comportements indéfinis
- des failles de sécurité
- des crashes
- des bugs difficiles à détecter

Adopter des règles strictes permet de prévenir ces risques et de produire un code fiable et industriel.

1.3 Contenu du document

Ce document couvre les aspects fondamentaux de la qualité en C :

- Compilation stricte et gestion des warnings
- Bon usage des types et conversions
- Validation des entrées utilisateur
- Gestion des erreurs et des retours de fonctions
- Manipulation sûre de la mémoire (pointeurs, tableaux, allocation)
- Écriture de code lisible et maintenable (noms, commentaires, structure)

1.4 Approche pédagogique

Chaque chapitre suit une structure simple et efficace :

- un exemple de **mauvais code**
- une version corrigée (**bon code**)
- une **explication** claire du problème et de la solution

Cette approche permet de comprendre rapidement les erreurs fréquentes et d'adopter les bons réflexes en situation réelle.

Chapitre 2

Compilation stricte (GCC)

2.1 Mauvais code

```
gcc fichier.c
```

2.2 Bon code

```
gcc -std=c11 -Wall -Wextra -Werror \  
-Wshadow -Wconversion -Wsign-conversion \  
-Wstrict-prototypes -Wimplicit-function-declaration \  
fichier.c -o app
```

2.3 Explication

Une compilation stricte permet de détecter les erreurs très tôt. Avec `-Werror`, tous les warnings deviennent bloquants, ce qui garantit un code sans ambiguïté ni comportement indéfini.

L'ajout d'autres paramètres permet d'améliorer la commande gcc afin d'y ajouter de nouvelles analyses. Cela permet ensuite au développeur de corriger et améliorer son programme plus efficacement.

Chapitre 3

Programme principal correct

3.1 Mauvais code

```
int main() {  
    /* ... */  
}
```

3.2 Bon code

```
int main(void) {  
    return 0;  
}
```

3.3 Explication

En C, `main()` sans `void` signifie que la liste des arguments n'est pas spécifiée. La forme `int main(void)` est plus claire, plus correcte et cohérente avec une compilation stricte.

Chapitre 4

Variables globales inutiles

4.1 Mauvais code

```
#include <stdio.h>

int count;

int main(void) {
    count = 5;
    printf("%d\n", count);
    return 0;
}
```

4.2 Bon code

```
#include <stdio.h>

int main(void) {
    int count = 5;
    printf("%d\n", count);
    return 0;
}
```

4.3 Explication

Les variables globales augmentent le couplage entre fonctions et rendent le code plus difficile à tester et à maintenir. Il faut préférer les variables locales tant qu'un partage global n'est pas justifié.

Chapitre 5

Variables non initialisées

5.1 Mauvais code

```
#include <stdio.h>

int main(void) {
    int x;
    if (x > 0) {
        printf("ok\n");
    }
    return 0;
}
```

5.2 Bon code

```
#include <stdio.h>

int main(void) {
    int x = 0;
    if (x > 0) {
        printf("ok\n");
    }
    return 0;
}
```

5.3 Explication

Lire une variable non initialisée provoque un comportement indéfini. Le compilateur peut parfois détecter ce cas, mais il faut surtout adopter une discipline systématique d'initialisation.

Chapitre 6

Valeurs de retour ignorées

6.1 Mauvais code

```
#include <stdio.h>

int main(void) {
    char buf[16];
    fgets(buf, sizeof buf, stdin);
    puts(buf);
    return 0;
}
```

6.2 Bon code

```
#include <stdio.h>

int main(void) {
    char buf[16];

    if (fgets(buf, sizeof buf, stdin) == NULL) {
        return 1;
    }

    puts(buf);
    return 0;
}
```

6.3 Explication

Ignorer la valeur de retour d'une fonction d'entrée/sortie empêche de détecter un échec. Il faut vérifier systématiquement les appels qui peuvent échouer.

Chapitre 7

Entrées utilisateur avec scanf

7.1 Mauvais code

```
#include <stdio.h>

int main(void) {
    int x;
    scanf("%d", &x);
    printf("%d\n", x);
    return 0;
}
```

7.2 Bon code

```
#include <stdio.h>

int main(void) {
    int x = 0;

    if (scanf("%d", &x) != 1) {
        return 1;
    }

    printf("%d\n", x);
    return 0;
}
```

7.3 Explication

`scanf` peut échouer si l'entrée ne correspond pas au format attendu. Sans test du retour, la variable peut rester indéterminée.

Chapitre 8

Conversion robuste avec strtoul

8.1 Mauvais code

```
#include <stdlib.h>

int main(void) {
    int x = atoi("12abc");
    return x;
}
```

8.2 Bon code

```
#include <errno.h>
#include <stdlib.h>

int parse_u32(const char *s, unsigned long *out) {
    char *end = NULL;
    unsigned long v;

    errno = 0;
    v = strtoul(s, &end, 10);

    if (errno != 0 || end == s || *end != '\0') {
        return 1;
    }

    *out = v;
    return 0;
}
```

8.3 Explication

`atoi` ne permet pas de distinguer une vraie conversion d'une conversion partielle ou invalide. `strtoul` associé à `errno` et au pointeur de fin permet une validation robuste.

Chapitre 9

Overflow signé

9.1 Mauvais code

```
#include <limits.h>

int add_bad(int a, int b) {
    return a + b;
}
```

9.2 Bon code

```
#include <limits.h>

int add_good(int a, int b, int *out) {
    if (out == NULL) {
        return 1;
    }

    if ((b > 0 && a > INT_MAX - b) ||
        (b < 0 && a < INT_MIN - b)) {
        return 1;
    }

    *out = a + b;
    return 0;
}
```

9.3 Explication

Le dépassement de capacité d'un entier signé provoque un comportement indéfini. Il faut vérifier les bornes avant l'opération.

Chapitre 10

Conversions signé / non signé

10.1 Mauvais code

```
#include <string.h>

int main(void) {
    int i = -1;
    size_t n = strlen("abc");
    return (i < n);
}
```

10.2 Bon code

```
#include <string.h>

int main(void) {
    size_t i = 0;
    size_t n = strlen("abc");
    return (i < n);
}
```

10.3 Explication

Comparer un entier signé négatif avec un type non signé force une conversion implicite piègeuse. Il faut choisir des types cohérents avec le domaine de valeurs attendu.

Chapitre 11

Masquage de variable

11.1 Mauvais code

```
int main(void) {  
    int x = 10;  
  
    if (x > 0) {  
        int x = 3;  
        (void)x;  
    }  
  
    return x;  
}
```

11.2 Bon code

```
int main(void) {  
    int x = 10;  
  
    if (x > 0) {  
        int y = 3;  
        (void)y;  
    }  
  
    return x;  
}
```

11.3 Explication

Le shadowing rend le code confus et favorise les erreurs de lecture. Il vaut mieux éviter de redéclarer une variable avec le même nom dans une portée interne.

Chapitre 12

Prototypes de fonctions

12.1 Mauvais code

```
int main(void) {  
    return add(2, 3);  
}  
  
int add(int a, int b) {  
    return a + b;  
}
```

12.2 Bon code

```
int add(int a, int b);  
  
int main(void) {  
    return add(2, 3);  
}  
  
int add(int a, int b) {  
    return a + b;  
}
```

12.3 Explication

Appeler une fonction sans prototype peut conduire à une signature supposée incorrecte. En compilation stricte, chaque fonction doit être déclarée avant usage.

Chapitre 13

Strict prototypes

13.1 Mauvais code

```
int foo();  
int main() {  
    return foo();  
}
```

13.2 Bon code

```
int foo(void);  
  
int main(void) {  
    return foo();  
}
```

13.3 Explication

En C, `int foo();` signifie que les arguments sont inconnus. `int foo(void);` exprime correctement qu'il n'y a aucun argument.

Chapitre 14

Affectation au lieu de comparaison

14.1 Mauvais code

```
int main(void) {  
    int x = 0;  
  
    if (x = 1) {  
        return 1;  
    }  
  
    return 0;  
}
```

14.2 Bon code

```
int main(void) {  
    int x = 0;  
  
    if (x == 1) {  
        return 1;  
    }  
  
    return 0;  
}
```

14.3 Explication

Utiliser `=` dans une condition modifie la variable au lieu de la comparer. C'est une erreur classique que les warnings de compilation permettent souvent de repérer.

Chapitre 15

Switch incomplet

15.1 Mauvais code

```
void foo(void);

void f(int s) {
    switch (s) {
        case 1:
            foo();
    }
}
```

15.2 Bon code

```
void foo(void);

void f(int s) {
    switch (s) {
        case 1:
            foo();
            break;
        default:
            break;
    }
}
```

15.3 Explication

Sans **break**, un cas peut tomber dans le suivant. Sans **default**, un état non prévu n'est pas explicitement géré.

Chapitre 16

Accès hors bornes dans les tableaux

16.1 Mauvais code

```
int get_bad(const int *a, unsigned len, unsigned idx) {  
    return a[idx];  
}
```

16.2 Bon code

```
int get_good(const int *a, unsigned len, unsigned idx, int *out) {  
    if (a == NULL || out == NULL) {  
        return 1;  
    }  
  
    if (idx >= len) {  
        return 1;  
    }  
  
    *out = a[idx];  
    return 0;  
}
```

16.3 Explication

Un accès hors bornes provoque un comportement indéfini. Il faut vérifier les bornes avant tout accès mémoire indexé.

Chapitre 17

Pointeurs nuls

17.1 Mauvais code

```
void set_bad(int *p) {  
    *p = 42;  
}
```

17.2 Bon code

```
int set_good(int *p) {  
    if (p == NULL) {  
        return 1;  
    }  
  
    *p = 42;  
    return 0;  
}
```

17.3 Explication

Déréférencer un pointeur nul provoque généralement un crash. Toute fonction manipulant un pointeur reçu de l'extérieur doit valider ce pointeur.

Chapitre 18

Allocation dynamique

18.1 Mauvais code

```
#include <stdlib.h>

int main(void) {
    int *p = malloc(10 * sizeof(int));
    p[0] = 1;
    free(p);
    return 0;
}
```

18.2 Bon code

```
#include <stdlib.h>

int main(void) {
    int *p = malloc(10 * sizeof(*p));

    if (p == NULL) {
        return 1;
    }

    p[0] = 1;
    free(p);
    p = NULL;
    return 0;
}
```

18.3 Explication

`malloc` peut échouer et retourner `NULL`. Après un `free`, remettre le pointeur à `NULL` réduit le risque d'usage accidentel ultérieur.

Chapitre 19

Erreur classique avec sizeof(pointer)

19.1 Mauvais code

```
#include <string.h>

int main(void) {
    const char *s = "hello";
    char buf[16];

    memcpy(buf, s, sizeof(s));
    return 0;
}
```

19.2 Bon code

```
#include <string.h>

int main(void) {
    const char *s = "hello";
    char buf[16];

    memcpy(buf, s, strlen(s) + 1);
    return 0;
}
```

19.3 Explication

Quand `s` est un pointeur, `sizeof(s)` donne la taille du pointeur, pas celle de la chaîne pointée. Il faut raisonner sur la donnée réelle, pas sur l'adresse.

Chapitre 20

Macros non sûres

20.1 Mauvais code

```
#define INC(x) x++  
#define SWAP(a,b) { int t=a; a=b; b=t; }
```

20.2 Bon code

```
#define INC(x) do { (x)++; } while (0)  
#define SWAP(a,b) do { int t = (a); (a) = (b); (b) = t; } while (0)
```

20.3 Explication

Une macro sans parenthèses ni structure sûre peut casser la logique du code appelant. Le motif `do { } while (0)` est la forme classique pour une macro multi-instruction.

Chapitre 21

Usage inutile de goto

21.1 Mauvais code

```
int f(int ok) {
    if (!ok) {
        goto err;
    }

    return 0;
err:
    return 1;
}
```

21.2 Bon code

```
int f(int ok) {
    if (!ok) {
        return 1;
    }

    return 0;
}
```

21.3 Explication

`goto` rend le flux d'exécution plus difficile à suivre. Il peut avoir des usages légitimes dans certains schémas de nettoyage, mais il faut l'éviter quand une structure plus simple suffit.

Chapitre 22

Commentaires utiles

22.1 Mauvais code

```
i++; /* incremente i */
```

22.2 Bon code

```
/* On limite les tentatives pour eviter une boucle infinie. */  
retry_count++;
```

22.3 Explication

Un bon commentaire explique l'intention ou la contrainte, pas une évidence syntaxique. Le lecteur doit comprendre pourquoi le code existe.

Chapitre 23

Paramètre inutilisé

23.1 Mauvais code

```
void cb(int x) {  
}
```

23.2 Bon code

```
void cb(int x) {  
    (void)x;  
}
```

23.3 Explication

Un paramètre non utilisé déclenche souvent un warning. Le cast en `(void)` documente explicitement que ce paramètre est volontairement ignoré.

Chapitre 24

Nommage cohérent

24.1 Mauvais code

```
#define max_retry 3
struct Person { int Age; };
int DoStuff();
```

24.2 Bon code

```
#define MAX_RETRY 3

typedef struct {
    int age;
} person_t;

int do_stuff(void);
```

24.3 Explication

Un nommage cohérent améliore fortement la lisibilité. Une convention simple peut être :

- macros en majuscules;
- types structurés avec suffixe `_t` si le projet l'autorise;
- fonctions et variables en minuscules avec underscore.

Chapitre 25

Style de code cohérent

25.1 Mauvais code

```
if(x>0){foo();}
```

25.2 Bon code

```
if (x > 0) {  
    foo();  
}
```

25.3 Explication

Un style homogène rend le code plus facile à lire, relire et maintenir. Peu importe la convention choisie, l'essentiel est d'être cohérent dans tout le projet.

Chapitre 26

Conclusion

Un code C de qualité professionnelle repose sur :

- une compilation stricte sans warning ;
- une gestion rigoureuse des types, des conversions et des prototypes ;
- des vérifications systématiques des retours, pointeurs et bornes ;
- un style cohérent et un nommage lisible ;
- une attention particulière à la robustesse mémoire et à la sécurité.

Ces pratiques permettent de produire un code plus fiable, plus maintenable et plus sûr dans la durée.